

MODULE 6 · SCHEDULING, RESOURCES & SCALING

Horizontal Pod Autoscaler

Scaling replica count automatically from observed CPU utilization

WHY IT MATTERS

Traffic doesn't wait for someone to run `kubectl scale`

A Deployment holds whatever replica count you gave it — 2 stays 2, whether traffic is quiet or spiking. Scaling by hand means someone watching a dashboard and reacting late.

The **Horizontal Pod Autoscaler (HPA)** is a controller that watches a metric — usually CPU % — and adjusts `spec.replicas` for you, continuously, without a human in the loop.

BY THE END YOU CAN

- Explain HPA's two hard prerequisites
- Read the metrics → HPA → scale loop
- Create an HPA both ways
- Diagnose an <unknown> TARGETS column

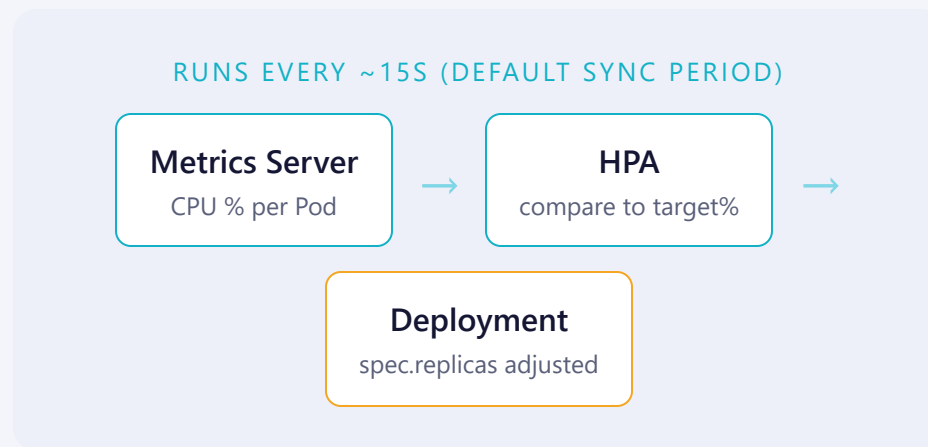
CONCEPT

One controller, watching CPU%, adjusting replicas

HPA doesn't touch Pods directly. It reads utilization from the **metrics API**, compares it to a target, and edits `spec.replicas` on the Deployment — the same field `kubectl scale` would edit.

The Deployment's own reconciliation loop (Lab 2.3) then does the actual work of creating or removing Pods.

HPA is a controller *on top of* a controller — it never talks to Pods directly.



Two prerequisites, non-negotiable

REQUIREMENT	WHY HPA NEEDS IT	MISSING IT →
Metrics Server installed	Serves live CPU/memory usage via the <code>metrics.k8s.io</code> API	HPA has no usage data to read at all
resources.requests.cpu on every container	Utilization % = usage ÷ requested amount — the request <i>is</i> the denominator	TARGETS shows <code><unknown>/50%</code>

MENTAL MODEL

CPU utilization is always *relative* — there's no percentage without a request to be a percentage **of**.

A target, a range, and a metric

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-nginx
  namespace: training
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-nginx    # what to scale
  minReplicas: 2      # floor
  maxReplicas: 10     # ceiling
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50 # % of request
```

scaleTargetRef names the Deployment, **minReplicas/maxReplicas** bound the range, and **metrics[].resource.target.averageUtilization** sets the % of request to hold.

The denominator lives on the target

```
resources:  
  requests:  
    cpu: 50m    # the denominator
```

This has to exist on the target Deployment's container, or HPA has nothing to divide usage by.

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF

```
kubectl autoscale deployment hpa-nginx --min=2 --max=10 --cpu-percent=50
```

Builds the HPA object for you, submits it now

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f hpa.yaml
```

You own the file; it's reviewable and re-appliable

THE BRIDGE TRICK

Generate the YAML without creating anything, then edit & keep it:

```
$ kubectl autoscale deployment hpa-nginx --min=2 --max=10 --cpu-percent=50 \
  --dry-run=client -o yaml
```

SEE IT

TARGETS climbs, REPLICAS follows

```
$ kubectl get hpa hpa-nginx -n training
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
hpa-nginx	Deployment/hpa-nginx	62%/50%	2	10	4

HPA won't chase every wobble: it ignores utilization within a **±10% tolerance band** around the target (here, 45–54%) to avoid flapping replicas up and down. CPU has to clear **~55%** before a 50%-target HPA scales up.

MENTAL MODEL

Formula the controller applies each loop: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentUtil} / \text{targetUtil})$

THE PUNCHLINE

No request means no percentage

Utilization is computed as $\text{usage} \div \text{request}$. Skip `resources.requests.cpu` on the container and there's nothing to divide by — HPA can't produce a percentage, so it doesn't guess.

The Deployment stays exactly where it is. No error, no event storm — just a controller quietly unable to act.

SAME HPA, ONE MANIFEST DIFFERENCE

requests set

45%/50%

requests missing

<unknown>/50%

Where people trip

- **No resources.requests.** The single most common cause of a stuck <unknown> TARGETS column.
- **Metrics Server not installed.** `kubectl top pods` failing means HPA has no data either — check that first.
- **Expecting instant scale-up.** The tolerance band deliberately absorbs small swings — that's not a bug.
- **Mixing `kubectl scale` with an active HPA.** The next reconcile loop overwrites your manual replica count.

Commands to keep at hand

```
$ kubectl top pods -n NS           # confirm Metrics Server works
$ kubectl get hpa -n NS           # TARGETS / MINPODS / MAXPODS / REPLICAS
$ kubectl describe hpa NAME -n NS # events + scaling decisions
$ kubectl autoscale deployment NAME --min=N --max=M --cpu-percent=P
$ kubectl patch hpa NAME --type=merge -p '{...}' # edit target/range live
$ kubectl delete hpa NAME -n NS    # stop autoscaling
```

Tip: stuck at <unknown>? Check requests first, Metrics Server second.

RECAP

HPA in one breath

- HPA reads CPU% from Metrics Server, then edits the Deployment's `spec.replicas`
- It needs both Metrics Server **and** `resources.requests` to compute a %
- A $\pm 10\%$ tolerance band prevents flapping around the target
- Missing requests $\rightarrow$ `<unknown>` targets, not an error

HANDS-ON

Now run Lab 6.4
[labs/6.4/](#)

Next: [6.5 — DaemonSets](#)